Adaptive Draft Sequence Length: Enhancing Speculative Decoding Throughput on PIM-Enabled Systems

Runze Wang†, Qinggang Wang*†, Haifeng Liu†, Long Zheng†, Xiaofei Liao†, Hai Jin† and Jingling Xue‡
†*National Engineering Research Center for Big Data Technology and System/Services Computing Technology and System Lab/Cluster and Grid Computing Lab, School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan, 430074, China*
‡*School of Computer Science and Engineering, University of New South Wales, Sydney, NSW 2052, Australia*
{rzwang, qgwang, hfliu, longzh, xfliao, hjin}@hust.edu.cn, jingling@cse.unsw.edu.au

2026 IEEE International Symposium on High Performance Computer Architecture (HPCA) | 979-8-3315-9302-5/26/$31.00 ©2026 IEEE | DOI: 10.1109/HPCA68181.2026.11408598

***Abstract*—Transformer-based *large language models* (LLMs) exhibit remarkable generative capabilities, but their inference throughput is limited by the autoregressive decoding process, which generates only one token per iteration. *Speculative decoding* mitigates this bottleneck by using a lightweight *draft language model* (DLM) to generate multiple draft tokens, which are then verified in parallel by a more accurate *target language model* (TLM). To accommodate the differing computational patterns of the DLM and TLM, prior work has leveraged heterogeneous systems combining xPUs and *processing-in-memory* (PIM) units to offload compute- and memory-intensive operators, respectively.**

However, existing systems often adopt a fixed draft sequence length, leading to excessive rejection of draft tokens during verification—especially under large-batch scenarios—resulting in redundant computation and reduced efficiency. This paper proposes a *runtime adaptive draft length* adjustment technique that dynamically tailors the draft length for each request by monitoring cumulative acceptance probabilities, thereby minimizing the generation and verification of invalid tokens. Yet, integrating adaptive draft lengths into existing PIM-enabled heterogeneous systems introduces two new challenges: (1) sequential execution of the DLM and TLM becomes inefficient due to synchronization bubbles caused by request-wise variability in draft lengths, and (2) static operator mappings become suboptimal as draft length variability alters operator arithmetic intensities dynamically. To address these issues, we introduce SADDLE, a PIM-enabled heterogeneous system designed to exploit adaptive draft lengths effectively. SADDLE incorporates two key mechanisms: (1) an *asynchronous speculative decoding pipeline* that decouples DLM prediction and TLM verification to reduce idle time, and (2) an *arithmetic intensity-aware operator scheduler* that dynamically assigns operators to the most suitable hardware units. Experimental results show that SADDLE achieves average speedups of 2.88× over a state-of-the-art GPU-only solution and 1.71× over the best-performing GPU+PIM baseline.

I. Introduction

Transformer-based *large language models* (LLMs) [3], [35], [47], [62] are revolutionizing the AI application ecosystem with their exceptional generative capabilities. They are widely deployed in a range of applications, including chatbots [8], [10], [36], code auto-completion [6], [31], [54], and complex reasoning services [12], [42].

*Corresponding author: Qinggang Wang (qgwang@hust.edu.cn).

LLM inference typically consists of two phases: *prefill* and *decoding*. In the prefill phase, the model processes all input tokens in the prompt (e.g., a request sequence) to generate the first output token. In the subsequent decoding phase, the model generates one output token per iteration in an autoregressive manner, where each token generation depends on the token produced in the previous iteration. As a result, generating T output tokens requires $T-1$ sequential decoding steps, which leads to low throughput and underutilization of compute resources during inference.

To improve inference throughput during the decoding phase, *speculative decoding* [5], [21], [29], [46], [60] has emerged as a promising technique. It first employs a lightweight *draft language model* (DLM) to predict the next d tokens for a given request, where d is a user-defined hyperparameter known as the **draft sequence length**. The original LLM, referred to as the *target language model* (TLM), then verifies these d draft tokens in parallel. During verification, if a draft token is rejected by the TLM, it is replaced with the TLM's own output token, and all subsequent draft tokens are discarded. Since DLM prediction is significantly faster than TLM verification, speculative decoding can substantially improve inference throughput—provided that a high proportion of DLM-generated tokens are accepted by the TLM.

The DLM prediction and TLM verification stages in speculative decoding exhibit distinct computational characteristics. The DLM still performs autoregressive decoding, predicting one draft token per iteration through memory-intensive matrix-vector multiplication operations. In contrast, the TLM verifies multiple draft tokens in parallel, relying on compute-intensive matrix-matrix multiplication operations. This computational heterogeneity has spurred research into xPU–PIM heterogeneous acceleration strategies for speculative decoding. Recent studies [16], [22], [38], [44] have explored PIM-enabled heterogeneous systems that coordinate computation-centric xPUs (e.g., GPUs and TPUs) with memory-centric PIM units (e.g., HBM-based devices with integrated PIM dies). These systems accelerate speculative decoding by statically mapping compute-intensive operators to xPUs and memory-intensive

operators to PIM units.

However, existing PIM-enabled heterogeneous systems [15], [22] fall short of fully exploiting the acceleration potential of speculative decoding. As shown in §III, increasing the number of concurrent requests (i.e., batch size) leads to lower throughput than systems using conventional autoregressive decoding without speculative decoding. This performance degradation arises from a large number of draft tokens being discarded during the verification phase, rendering the computation spent on their generation and verification ineffective. While such redundant computation may be tolerable under small batch sizes, it becomes increasingly detrimental as the batch size grows—consuming valuable hardware resources that could otherwise be used for effective inference and ultimately resulting in a significant drop in overall throughput.

Our investigation reveals that the root cause of redundant computation in existing systems is the use of a fixed draft sequence length. In practice, *draft token acceptance rates*—the ratio of accepted to predicted tokens—vary significantly across models, datasets, and batch sizes [25], [57]. As a result, the optimal draft length should adapt dynamically at runtime. When the optimal draft length is shorter than the fixed one, the DLM generates superfluous tokens that are likely to be rejected, resulting in wasted computation and reduced inference throughput. Conversely, when the optimal draft length exceeds the fixed one, the DLM could have generated more tokens that the TLM would accept in a single verification round. However, the fixed-length constraint forces repeated interruptions of DLM prediction and incurs multiple TLM verification rounds, reducing parallelism and further degrading throughput.

To enable high-throughput speculative decoding on PIM-enabled heterogeneous systems, we propose leveraging an *adaptive draft sequence length* to allow flexible, on-demand draft token generation that addresses both redundant computation and degraded parallelism. However, incorporating adaptive draft lengths introduces three key challenges.

First, determining the optimal draft sequence length is non-trivial, as it depends on dynamic factors such as the model, dataset, and input request. Identifying an appropriate length for each request on-the-fly must incur minimal runtime overhead.

Second, variable draft lengths across requests combined with the sequential execution of the DLM and TLM can lead to severe pipeline bubbles. Specifically, the TLM must wait for the DLM to complete its predictions before starting verification, and the DLM must stall until verification finishes before proceeding to the next round. Within a batch, requests with shorter draft lengths are forced to wait for those with longer ones to finish DLM prediction before triggering TLM verification, further exacerbating idle time and increasing overall inference latency.

Third, the varying draft lengths dynamically alter the arithmetic intensity of operators, making static operator-to-device mappings suboptimal. For example, SpecPIM [22] determines its operator mapping through offline analysis based on initial configuration and does not adjust during execution. When draft lengths change at runtime, such static mapping can assign compute-intensive operators to PIM units or memory-intensive operators to xPUs, resulting in inefficient execution.

This paper develops a PIM-enabled heterogeneous system for high-throughput speculative decoding that supports adaptive draft sequence lengths while mitigating the associated performance challenges. We introduce **SADDLE**, a PIM-enabled heterogeneous System that leverages ADaptive Draft sequence LEngths to enhance speculative decoding throughput.

SADDLE embodies three key technical innovations. First, it features a runtime adaptive draft length adjustment mechanism that dynamically tunes the draft length for each request based on its cumulative acceptance probability, thereby reducing invalid draft token generation and verification while preserving parallelism. Second, SADDLE employs an asynchronous speculative decoding pipeline that decouples DLM prediction from TLM verification to alleviate pipeline stalls. Third, it integrates an arithmetic intensity-aware operator scheduling strategy that continuously monitors operator arithmetic intensity and dynamically maps operators to the most suitable hardware units, thereby maximizing the acceleration potential of the heterogeneous architecture.

In summary, this paper makes the following contributions:

- We identify the intrinsic cause of suboptimal speculative decoding throughput in existing PIM-enabled heterogeneous systems and elucidate the key challenges these systems face when adopting adaptive draft sequence lengths.
- We propose *SADDLE*, a heterogeneous system that combines GPUs with HBM-based PIM devices to enable high-throughput speculative decoding. SADDLE dynamically adjusts the draft sequence length per request to reduce invalid token generation and verification, representing the first use of adaptive draft lengths in PIM-enabled heterogeneous systems.
- SADDLE incorporates two novel mechanisms: an asynchronous speculative decoding pipeline to mitigate pipeline stalls, and an arithmetic intensity–aware operator scheduler that maximizes the acceleration potential of the heterogeneous architecture.
- We evaluate SADDLE on diverse LLM models and datasets, demonstrating that it outperforms the state-of-the-art GPU-only and GPU+PIM systems, achieving average throughput improvements of 2.88× and 1.71×, respectively.

The rest of this paper is organized as follows. §II introduces the necessary background on speculative decoding and PIM-enabled heterogeneous systems. §III presents a comprehensive performance analysis of speculative decoding on a PIM-enabled heterogeneous system and motivates our design. §IV details the proposed SADDLE architecture along with its three key components. §V reports and analyzes our experimental results. §VI reviews related work, and §VII concludes.

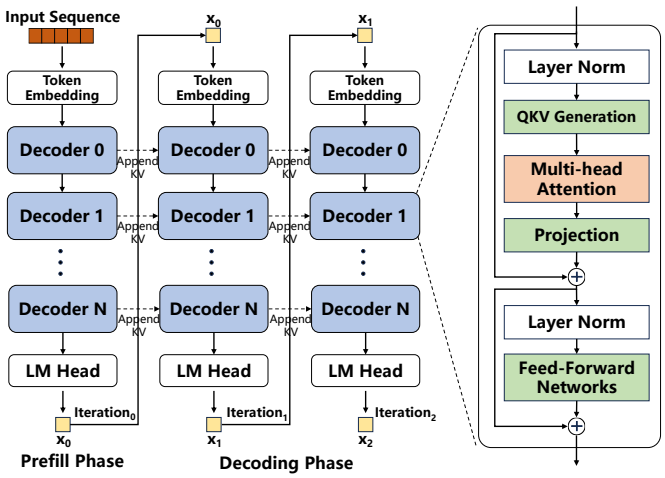

Fig. 1. Transformer-based LLM structure

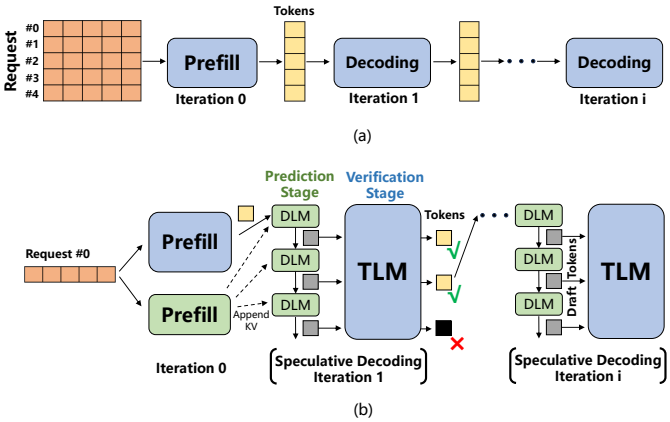

Fig. 2. Batching and speculative decoding in LLM inference. (a) Batching enables request-level parallelism. (b) Speculative decoding enables token-level parallelism via draft token generation (DLM) and parallel verification (TLM).

II. Background

In this section, we review transformer-based LLM inference, its parallelism optimizations, and PIM-enabled heterogeneous acceleration solutions.

A. Transformer-based LLM Inference

As shown in Figure 1, mainstream LLMs are composed of multiple Transformer decoder [48] layers and operate in two phases: the *prefill* phase and the *decoding* phase. In the prefill phase, the LLM processes the entire input sequence in parallel to generate the first output token. In the decoding phase, the LLM takes the concatenation of the original prompt and all previously generated tokens as input to generate the next token. This process repeats until an end-of-sequence token is produced. Since only one token is generated per iteration, autoregressive decoding is inherently sequential and results in low throughput.

Each decoder layer primarily consists of three kernels: QKV (query, key, and value) generation, *multi-head attention* (MHA), and *feed-forward networks* (FFN). These kernels fall into two categories: QKV generation and FFN are *fully connected* (FC) operators, while MHA represents the attention operator. All these operators rely on *general matrix-vector multiplication* (GEMV). During MHA execution, keys and values from all previously generated tokens must be accessed to compute the next token. To avoid redundant KV computations, a caching mechanism—commonly known as the KV cache [19], [58]—is employed to store previously generated keys and values for reuse in future decoding iterations.

B. Parallel Optimization Techniques in LLM Inference

In production LLM inference systems, users issue requests continuously. To overcome the low throughput of serial autoregressive decoding and ensure timely responses, two parallel optimization techniques are introduced: *batching* and *speculative decoding*. These techniques enable concurrent decoding of multiple tokens, improving overall inference throughput by generating more than one token per decoding iteration.

Batching. As illustrated in Figure 2(a), batching [1], [23], [40], [58], [63] allows a single decoding iteration to generate multiple tokens in parallel across different user requests. In the FC operator, which multiplies each token's activation vector with a shared weight matrix, batching transforms multiple GEMVs from different requests into a single *general matrix-matrix multiplication* (GEMM), enabling efficient *request-level parallelism*.

In the attention operator, the query vector of the current token is multiplied by a matrix composed of the key vectors from all previous tokens, followed by another matrix-vector multiplication with the corresponding value vectors. These operations require frequent access to the KV cache, and in large-batch scenarios, the resulting memory traffic can become a bottleneck, constraining LLM inference by memory bandwidth.

Speculative Decoding. As shown in Figure 2(b), speculative decoding [4], [24], [60] introduces a parallel decoding mechanism comprising two stages: *serial draft token generation* (prediction) and *parallel draft token verification* (verification). First, a lightweight *draft language model* (DLM) rapidly predicts the next d draft tokens through d sequential decoding iterations. These tokens are then simultaneously verified by the *target language model* (TLM), enabling *token-level parallelism*.

Draft tokens that pass verification are accepted as output. If a token is rejected, it and all subsequent draft tokens are discarded. The TLM corrects the first erroneous token and uses it as the input for the next speculative iteration. Notably, verifying multiple tokens with the TLM is only slightly slower than generating a single token with the original LLM. Thus, when draft tokens are accepted, speculative decoding can produce multiple output tokens in one iteration. Even in the worst case, it still yields one corrected token.

Specifically, during the prediction stage, the DLM iteratively generates d draft tokens $(x_1, x_2, \ldots, x_d)$, while in the verification stage, the TLM simultaneously computes the corresponding probabilities $p_T(x_1), p_T(x_2), \ldots, p_T(x_d)$. A draft token

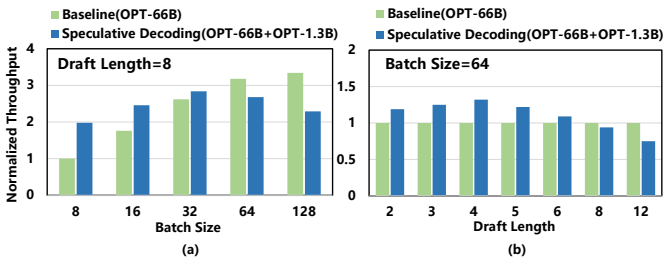

Fig. 3. Inference throughput of SpecPIM (OPT-66B+OPT-1.3B) versus the baseline (OPT-66B) under (a) varying batch sizes (draft length = 8) and (b) varying draft lengths (batch size = 64). Throughput is normalized to the baseline. SpecPIM's throughput peaks at $d = 4$, but degrades with longer drafts due to increased token rejection and wasted computation.

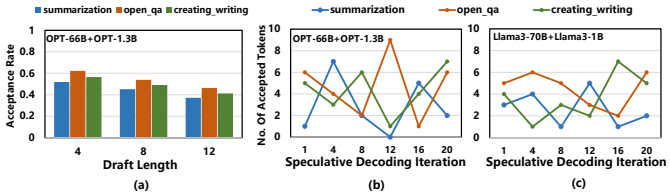

Fig. 4. (a) Token acceptance rates of the OPT-66B + OPT-1.3B model pair across three task categories in the Dolly dataset under different fixed draft lengths. (b) Per-iteration distribution of accepted tokens during speculative decoding for each task category using OPT-66B + OPT-1.3B. (c) Same as (b) using LLaMA3-70B + LLaMA3-1B. The results reveal dynamic and task-specific variations in optimal draft lengths across iterations and model pairs.

x_i is accepted if the DLM probability $p_D(x_i)$ is less than or equal to the TLM probability $p_T(x_i)$; otherwise, it is rejected with probability $1 - \frac{p_T(x_i)}{p_D(x_i)}$ and resampled from a normalized distribution proportional to $\max(0, p_T(x_i) - p_D(x_i))$. Moreover, if all d draft tokens are accepted, the TLM proceeds to generate an additional token x_{d+1}, which then serves as the input for the next round of DLM prediction. This prediction-verification cycle repeats until the end-of-sequence token is produced.

C. PIM-Enabled Heterogeneous Systems for LLM Inference

Recent studies [15], [22], [38], [59], [64] have explored PIM-based acceleration for LLM inference. By embedding *processing elements* (PEs) within memory and offloading memory-bound operators to them, PIM leverages bank-level parallelism to deliver higher internal bandwidth and reduces data movement by transferring only the final results.

AttAcc [38] accelerates autoregressive decoding by offloading attention operators to HBM-based PIM units, while executing other operators on the GPU. NeuPIM [16] and IANUS [44] integrate NPUs with PIM units to build heterogeneous systems for LLM inference. They employ dual-buffering and PIM-aware scheduling, respectively, to enable concurrent execution across NPUs and PIM units.

SpecPIM [22] is the first PIM-enabled heterogeneous system tailored for speculative decoding. It performs offline analysis to assign different operators to either PIM units or xPUs based on the initial parameter configuration (e.g., a fixed draft sequence length) before execution and maintains the operator-to-device mapping scheme unchanged throughout inference. Invoking offline analysis to reassign operators in response to dynamic changes in configuration parameters introduces non-negligible runtime overhead, which can easily outweigh the potential performance gains from remapping.

PAPI [15] supports operator remapping in response to changes in batch size, enabling flexible hardware configurations. However, it is primarily designed for a single model—the TLM—and falls short of achieving end-to-end acceleration for speculative decoding. In particular, it does not address the idle time introduced by the sequential execution between the DLM and TLM.

III. Motivation

In this section, we analyze speculative decoding throughput on a PIM-enabled heterogeneous system, motivating the design of our approach.

A. Bottleneck Analysis of Speculative Decoding on Existing PIM-Enabled Heterogeneous Systems

We analyze inefficiencies in existing solutions using SpecPIM [22], a state-of-the-art PIM-enabled heterogeneous system with four NVIDIA A100 GPUs, each paired with five HBM-PIM devices [38]. We adopt OPT-66B [61] as the TLM and OPT-1.3B as the DLM (denoted OPT-66B+OPT-1.3B). The baseline uses the same hardware as SpecPIM but runs conventional autoregressive decoding with OPT-66B. By default, we set the batch size to 64, draft length to 8, and use the Dolly [7] dataset for evaluation.

Figure 3(a) shows the inference throughput of SpecPIM and the baseline system under varying batch sizes. SpecPIM's throughput first increases with batch size but then decreases, and notably, falls below that of the baseline with standard autoregressive decoding when the batch size reaches 64.

The root cause of the throughput degradation lies in the *fixed draft sequence length*. Figure 3(b) shows SpecPIM's throughput at batch size 64 under varying draft lengths. Throughput peaks at $d = 4$, beating the baseline by 1.41×, but degrades as d increases, falling below the baseline at $d = 8$.

While longer draft lengths theoretically offer higher TLM parallelism, they lower token acceptance rates. As shown in Figure 4(a), acceptance rates across `creative_writing`, `summarization`, and `open_qa` drop with longer drafts, leading to more discarded tokens that waste compute and memory. Conversely, short draft lengths (e.g., $d = 2$) produce too few tokens per prediction stage, increasing verification frequency and reducing TLM parallelism. For example, $d = 2$ yields 1.12× lower throughput than $d = 4$.

Moreover, while an appropriate draft length can be selected for a given batch size (e.g., $d = 4$ in Figure 3(b)), applying the same fixed draft length to all requests in the batch results in suboptimal throughput due to wasted computation (Figure 5(a)). This inefficiency arises because, in each speculative decoding iteration—consisting of a DLM prediction stage followed by a TLM verification stage—each request has its

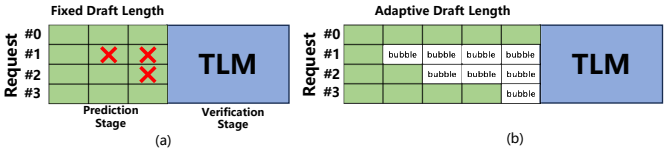

Fig. 5. Inefficiencies in speculative decoding pipeline designs: (a) Fixed draft lengths lead to wasted computation when draft tokens are later rejected by the TLM (marked by X); (b) A naive realization of adaptive draft length introduces pipeline bubbles due to synchronization delays when requests in the same batch have varying draft lengths.

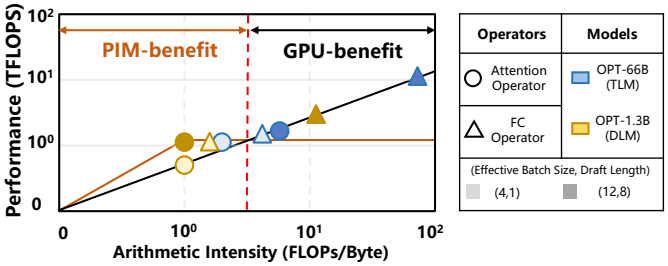

Fig. 6. Roofline analysis of speculative decoding operators on OPT-66B (TLM) + OPT-1.3B (DLM) under varying draft sequence lengths and effective batch sizes. The arithmetic intensity (FLOPs/Byte) and performance (TFLOPS) of the attention and FC operators shift significantly across configurations, highlighting transitions in bottlenecks (PIM compute-bound vs. GPU bandwidth-bound) and the corresponding optimal hardware execution targets.

own optimal draft length, defined as the maximum number of draft tokens that can be accepted in that iteration.

Figure 4(b) presents the optimal draft lengths across different speculative decoding iterations for three request tasks. For instance, in the `creative_writing` task, the optimal draft length decreases from 5 in the first iteration to 1 by the 12^{th} iteration. Consequently, using a fixed draft length across all requests and decoding iterations leads to either redundant computation (when the fixed value exceeds the optimal length) or reduced parallelism (when it falls short), both of which degrade overall inference efficiency.

B. Our Proposal: Adaptive Draft Sequence Length for PIM-Enabled Heterogeneous Systems

Through the above comprehensive analysis, a seemingly intuitive solution for achieving high-throughput speculative decoding is to adopt *adaptive draft sequence lengths* in existing PIM-enabled heterogeneous systems. This would enable flexible and on-demand draft token generation to mitigate redundant computation and parallelism degradation. However, realizing this goal poses three significant challenges.

First, several dynamically changing factors—such as the request task and model architecture—jointly determine the optimal draft sequence length. As shown in Figure 4(b), under the OPT-1.3B model, the three request tasks exhibit different optimal draft lengths across speculative decoding iterations. Moreover, Figure 4(c) reveals that switching from OPT-1.3B to LLaMA3-1B alters the optimal draft lengths for the same tasks and decoding iterations. These observations demonstrate that both task semantics and model choice significantly influence the optimal draft sequence length, creating a vast search space that is infeasible to exhaustively explore at runtime.

Second, using adaptive draft sequence lengths together with the inherently sequential execution of the DLM and TLM introduces severe pipeline bubbles. In standard speculative decoding, the DLM first performs d decoding iterations to generate d draft tokens per request, after which the TLM verifies all tokens in parallel. However, as illustrated in Figure 5(b), when requests in a batch have different draft lengths, those with shorter drafts must wait until the DLM completes the longest draft before verification can proceed. This synchronization introduces significant idle time, inflating the overall inference latency.

Third, dynamically varying draft lengths alter the arithmetic intensity of operators, making the static operator-to-device mappings in existing systems [38] suboptimal. To examine this effect, we conduct a roofline analysis on OPT-66B + OPT-1.3B under varying draft lengths and effective batch sizes, as shown in Figure 6. Here, *effective batch size* refers to the number of active requests at each stage of draft token generation. This number decreases as requests complete early and increases again when completed requests pass verification and resume prediction. The analysis shows that operator arithmetic intensity (FLOPs/byte) varies substantially with these parameters, thereby shifting the optimal hardware execution target. For instance, as the draft length increases from 1 to 8, the GPU outperforms PIM units for the TLM attention operator, even though the operator remains memory-bound on the GPU.

A more complex case arises when a batch starts with 12 requests, each with a distinct draft length. Once 8 shorter requests complete token generation, the effective batch size for subsequent DLM computation drops to 4. This change lowers the arithmetic intensity of the DLM's FC operator, transitioning it from GPU bandwidth-bound to PIM compute-bound. Hence, the optimal hardware target shifts from GPU to PIM. Static mappings thus result in inefficiencies, and repeatedly invoking offline remapping tools incurs high overhead, which may outweigh any gains from re-optimization.

IV. The SADDLE Architecture

In this section, we first present the overall architecture of SADDLE, then describe its workflow, and finally elaborate on its three key techniques.

A. Architecture Overview

Figure 7(a) illustrates the overall architecture of SADDLE, which consists of a host, a hardware manager, and multiple SADDLE PIM devices to enable end-to-end acceleration of speculative decoding with adaptive draft sequence lengths. The host communicates with the PIM devices through high-speed interconnects (e.g., CXL [45] or NVLink [33]). The hardware manager dynamically adjusts draft sequence lengths, coordinates pipeline scheduling between the DLM and TLM, and performs runtime operator scheduling. Model weights and KV caches for both the TLM and DLM reside on the

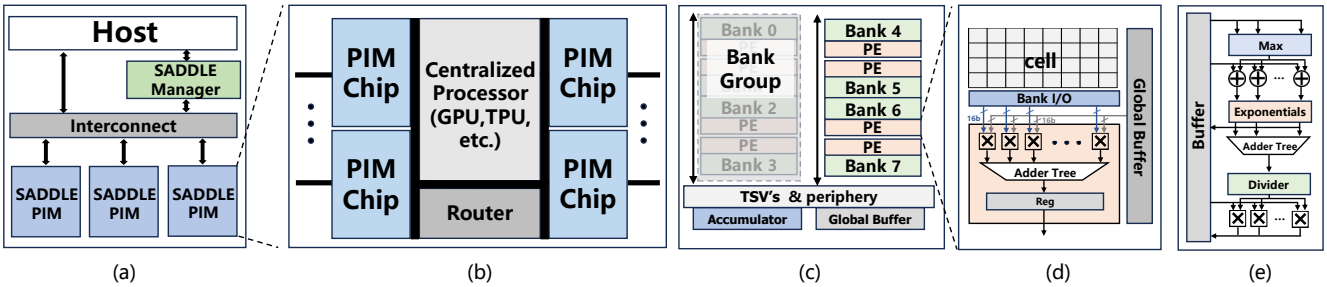

Fig. 7. Overview of the SADDLE computing system, consisting of a host, SADDLE Manager, and multiple SADDLE PIM devices interconnected via high-speed links (e.g., CXL or NVLink). (b) Architecture of a SADDLE PIM device, including a centralized processor (e.g., GPU or TPU), router, and multiple PIM chips. (c) Internal structure of a PIM chip based on the HBM-PIM architecture, showing the pseudo-channel (pCH) with PE-attached banks, global buffer, and accumulator. (d) Datapath architecture of a Processing Element (PE) supporting parallel matrix computations. (e) Specialized functional unit (SFU) for softmax and related operations such as normalization.

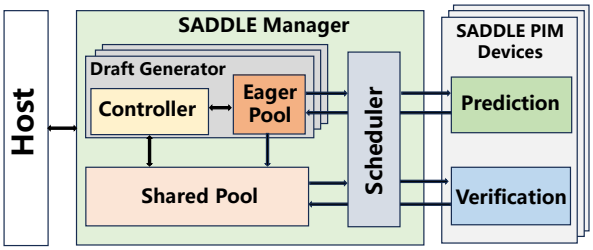

Fig. 8. Overview of the SADDLE Manager

PIM devices. The number of PIM devices can be scaled to accommodate varying model sizes and inference workloads.

SADDLE Manager. Leveraging pipeline parallelism, the SADDLE Manager assigns a *Draft Generator* to each micro-batch, consisting of a *Controller* and an *Eager Pool*. As shown in Figure 8, the Manager also includes a *Shared Pool* and a *Scheduler*. The Controller adaptively adjusts the draft sequence length for each request in the batch and determines whether a request already has draft tokens undergoing verification. If not, the newly generated token is placed in the Shared Pool; otherwise, it is temporarily held in the Eager Pool. All the tokens in the Shared Pool are verified in parallel, while the Scheduler performs dynamic operator remapping to appropriate hardware resources (PIMs or GPUs) to ensure optimal execution.

SADDLE PIM Devices. As shown in Figure 7(b), each SADDLE PIM device consists of a centralized processor, a router, and multiple PIM chips. In our design, the centralized processor is a GPU, although other high-performance processors (e.g., TPUs) optimized for compute-intensive operators can also be used. To maximize throughput, we employ HBM-based PIM chips due to their high bandwidth. The router handles data transfers between the PIM devices.

PIM Chips. Each PIM chip is based on the commercial HBM-PIM architecture [20]. It comprises a buffer die stacked beneath eight DRAM dies, all vertically integrated using *through-silicon vias* (TSVs). Each DRAM die exposes eight independently operable *pseudo channels* (pCHs), and each pCH contains four *bank groups* (BGs), with each bank group comprising four banks. Figure 7(c) illustrates a subset of a pCH, which includes a global buffer and an accumulator.

PEs. Each bank is paired with a dedicated PE. As illustrated in Figure 7(d), each PE consists of 16 FP16 multipliers, 16 FP16 adders, and associated registers. It processes two 256-bit operands per cycle, sourced from the bank's local row buffer and the pCH's global buffer. This architecture has been validated as feasible in prior work [38], [64]. While adding more arithmetic units could further increase bandwidth, doing so would violate HBM's stringent area and power constraints, as these units are fabricated using a DRAM process rather than a logic-optimized one. All PEs within a pCH operate in parallel across banks, thereby maximizing internal memory bandwidth.

SFUs. In addition to GEMV, Transformer layers involve operations such as residual addition, softmax, layer normalization, and activation functions. To support these operations, we integrate a *Specialized Functional Unit* (SFU) on the buffer die of each HBM stack, as depicted in Figure 7(e). The SFU is designed to efficiently handle these non-matrix operations, complementing the PEs and enhancing overall support for end-to-end Transformer inference.

B. Execution Flow and Data Mapping

As LLM parameters and KV cache sizes continue to grow, their memory demands can easily exceed the capacity of a single device. Pipeline parallelism [2], [17], [30], [56], [63] is a widely adopted technique for distributing LLM inference workloads. Following prior work [22], [30], [63], we partition PIM devices into S groups and assign model layers to S pipeline stages, each responsible for sequentially processing its assigned layers. A batch is divided into slightly more than S *micro-batches*—each comprising a subset of requests in the batch—to fully occupy all pipeline stages, thereby maximizing resource utilization and hiding communication overhead.

As a caveat, this pipelined execution operates on micro-batches, with each stage processing one at a time. Hereafter, references to micro-batches are made in this context.

Due to the distinct computational characteristics of DLM and TLM operators in speculative decoding—along with dy-

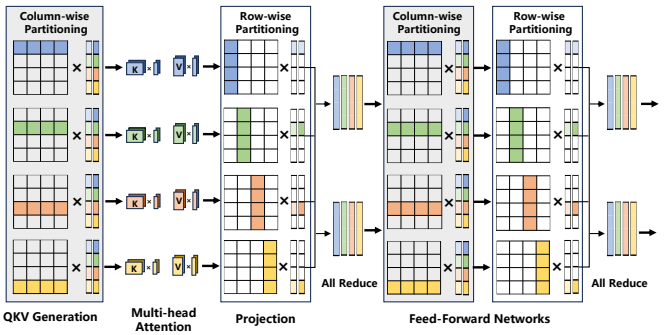

Fig. 9. Data sharding and execution flow of the decoder. QKV and first FFN layers use column-wise partitioning, while attention projections and second FFN layers use row-wise partitioning. AllReduce consolidates intermediate results to enable parallel execution with minimal communication.

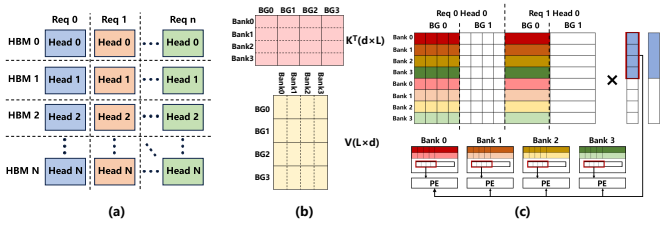

Fig. 10. Mapping strategy for the KV cache. (a) Each attention head is assigned to a specific HBM stack, with heads from different requests potentially sharing the same stack. (b) $K^\top$ is partitioned column-wise at the pCH and BG levels, and row-wise at the bank level. V is partitioned row-wise at the pCH and BG levels, and column-wise at the bank level. (c) Bank-level layout and dataflow for computing attention: K^T and V matrices are partitioned across bank groups and striped across banks to balance load and maximize row buffer reuse and internal memory bandwidth. The q vector is broadcast from the global buffer to all banks in a group for parallel processing.

namically varying draft lengths and effective micro-batch sizes—the optimal hardware accelerator for each operator may shift at runtime. This necessitates a carefully designed data mapping scheme that encompasses both the placement of model weight parameters in memory banks and the allocation of reserved space for KV caches used in attention computations, as these caches dynamically expand with each generated token.

To maximize inference throughput, the data mapping scheme must meet three key objectives [53]. First, it should exploit data locality by co-locating weights so that activations from multiple requests in a micro-batch access the same bank or buffer, improving reuse [55]. Second, it should balance the workload by evenly assigning non-reusable operators across bank groups or pCHs, thereby improving parallelism and bandwidth utilization. Third, it should reduce inter-PIM-device communication by minimizing data migration between operators or memory banks, lowering both latency and energy.

Figure 9 illustrates SADDLE's execution flow and data mapping for the QKV and FFN layers, as detailed below.

Weight Matrix Partitioning. Weight matrices can be partitioned along rows or columns. For QKV generation, as shown in Figure 9, we apply row-wise partitioning (i.e., by attention heads) to keep each head's weights contiguous. The input dimension equals the model's hidden size d_{model}, and each head generates an output of size $d_h = d_{\text{model}}/n$, where n is the number of heads. This layout allows the QKV outputs to be directly consumed by the MHA stage, where each head computes a d_h-dimensional output vector.

In the subsequent projection step, the weight matrix is partitioned column-wise, with each partition producing a partial output vector of size d_{model}. An all-reduce operation then aggregates these partial results into a complete output vector before feeding into the FFN. The FFN's two FC layers adopt a similar scheme: the first FC layer uses column-wise partitioning, while the second uses row-wise. This alternating partitioning maximizes inter-PIM-device bandwidth utilization while preserving computational continuity within each operator.

KV Cache Mapping. Figure 10(a) illustrates our KV cache mapping strategy. Each attention head is assigned to a specific HBM stack, and multiple heads from different requests may share the same stack. As attention heads operate independently, no inter-head communication is required.

K^T matrices are first partitioned column-wise across different BGs. Within each BG, K^T matrices are further striped row-wise across all banks. In contrast, V matrices are first partitioned row-wise across BGs and further striped column-wise across all banks within each BG. This partitioning strategy balances load and maximizes row buffer reuse and internal memory bandwidth.

To fully utilize peak internal bandwidth, our design maximizes row buffer usage by ensuring broad data consumption from it. Figure 10(c) illustrates an example of computing the multiplication between the q and v vectors: the q vector is broadcast from the global buffer to all banks in the bank group, enabling parallel computation across banks.

C. Adaptive Draft Length Adjustment

The goal of adjusting the draft sequence length for each request is to closely approximate its optimal value. However, determining this optimal length *before* the prediction stage begins is inherently difficult. Fortunately, it can be estimated *adaptively* during prediction by monitoring the cumulative probability of the generated tokens. This allows the system to decide when to halt drafting early, thereby avoiding unnecessary computation on tokens that are likely to be rejected.

Specifically, at each drafting iteration t, the Controller first computes the DLM's predicted distribution over next tokens, denoted $P(x_t \mid x_{<t})$ and samples a token x_t from it. Let $p_t := P(x_t \mid x_{<t})$ denote the probability of the sampled token. The cumulative probability of the current draft sequence up to step t is then computed as $H_t = \prod_{i=1}^{t} p_i$.

As the draft sequence grows, the cumulative probability H_t typically decreases. To avoid drafting tokens with low confidence, we introduce a threshold-based stopping mechanism: if H_t drops below a predefined threshold τ after sampling a token, the Controller terminates drafting for that request.

The threshold τ is learned offline using a validation set of real-world input sequences. For each request, we run the full prediction-verification pipeline, recording H_j and the final verification outcome at each draft step j. We estimate the conditional success rate curve over H_j and identify the 20% interval that yields the highest average draft length while maintaining at least a 90% verification success rate. These empirical criteria strike a balance between throughput and correctness. A discrete grid search over this interval selects the optimal τ.

At runtime, τ can be further adjusted dynamically. Lowering τ under light system load allows longer drafts, boosting parallelism and throughput. This adaptive strategy ensures that only high-confidence tokens are produced. If a token has low probability, the resulting drop in H_t triggers early stopping, avoiding wasteful computation on likely-to-be-invalid tokens.

The Controller integrates a softmax unit, multipliers, and comparators to enable fast, low-latency decisions for dynamically determining draft lengths.

Despite the benefits of adaptive draft lengths, applying them independently to each request in a micro-batch introduces synchronization challenges. As illustrated in Figure 5(b), requests that complete drafting early are forced to idle until the slowest request finishes, delaying the transition to the verification stage. While this synchronization barrier enables more draft tokens to be verified in parallel, it also causes early-finished requests to incur additional waiting time, increasing their inter-token latency and partially offsetting the gains from adaptive drafting.

To address this issue, we introduce a cross-micro-batch *Shared Pool* for draft tokens. Rather than verifying tokens immediately after generation, each micro-batch's Draft Generator accumulates them in the Shared Pool. When the total reaches the GPU's parallel verification capacity C or the GPU becomes idle, all tokens are dispatched to the TLM for verification. This *cross-micro-batch verification* strategy improves utilization and reduces idle time. The Shared Pool leverages *Content Addressable Memory* (CAM) for efficient storage, indexing, and retrieval.

Simple requests are granted longer draft lengths, while more complex ones receive shorter lengths. This is because simple requests tend to produce high-confidence tokens that are more likely to pass verification, making it efficient to generate longer drafts for them. In contrast, complex requests are more prone to verification failures, so limiting their draft length helps avoid unnecessary computation and reduces wasted effort. The Manager adopts a greedy strategy to prioritize and verify draft tokens that are most likely to pass verification across all requests. This approach maximizes effective system throughput and prevents any single request with an excessively long draft from delaying verification for the entire micro-batch, thereby ensuring low latency and high responsiveness.

D. Prediction-Verification Decoupled Asynchronous Pipeline

While our cross-micro-batch verification mechanism enhances parallelism, it may inadvertently exacerbate idle periods. Since tokens from multiple requests are aggregated into the Shared Pool, the pool can quickly reach its verification threshold after collecting only a few draft tokens per request. This premature saturation forces some requests to stop drafting earlier than optimal, limiting the depth of draft generation. For instance, a request that could ideally generate five draft tokens may be compelled to proceed to verification after producing only two, due to the Shared Pool filling up. As a result, the request fails to fully exploit its drafting potential, reducing overall efficiency.

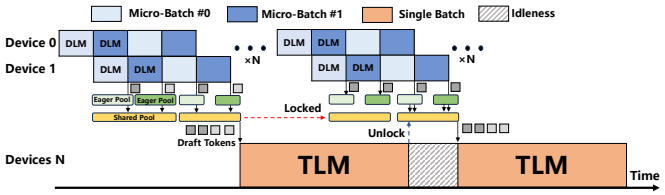

Fig. 11. Example execution timelines of synchronous pipeline

To this end, we decouple the prediction and verification stages to break their strict sequential dependency, allowing requests to continue drafting tokens even while verification is underway. This mechanism relies on an optimistic assumption: all draft tokens currently under verification will be accepted. Based on this assumption, any new tokens generated by the DLM during the verification stage are treated as valid and will be seamlessly incorporated into the next round of TLM verification.

Figure 11 illustrates the parallel execution of DLM prediction and TLM verification. Initially, draft tokens generated for each micro-batch are directly inserted into the Shared Pool. Once the Shared Pool is full, the TLM verification is invoked to validate all draft tokens currently stored in the Shared Pool. Meanwhile, for any request whose cumulative acceptance probability H_t remains above the threshold τ, DLM prediction continues to generate new draft tokens, which are temporarily stored into the Eager Pool. After TLM verification completes, draft tokens in the Eager Pool are processed as follows. If all previously generated draft tokens of a request are accepted by the TLM, the newly generated draft tokens for that same request are migrated from the Eager Pool into the Shared Pool. In contrast, if a draft token of a request is rejected by the TLM, all newly generated draft tokens for that request are discarded, and DLM prediction is resumed using the TLM-corrected token as the new input. Note that whenever the Shared Pool is not full, newly generated draft tokens bypass the Eager Pool and are inserted directly into the Shared Pool. In this way, the DLM prediction executes concurrently with the TLM verification, minimizing idle time and improving overall hardware resource utilization. Note that draft token migration between the Shared Pool and the Eager Pool involves only lightweight on-chip memory operations and therefore incurs negligible performance overhead. Moreover, since these cached draft tokens are refreshed after each verification iteration, the required capacities of both the Eager Pool

and the Shared Pool remain extremely small (∼1 KB).

E. Arithmetic Intensity-Aware Operator Scheduling

With dynamic draft lengths and cross-micro-batch verification, the arithmetic intensity of each operator fluctuates at runtime due to varying draft lengths and batch sizes (Figure 6). Additionally, concurrent execution of the target and draft models further impacts operator mapping. To address these dynamics, we now establish an initial execution mapping scheme based on the system's average workload profile.

The peak compute performance and memory bandwidth of each hardware device are known in advance. Based on this information, the Scheduler initially assigns the DLM's attention operators and the TLM's FC operators to fixed hardware resources. For the DLM, each iteration generates exactly one token per request, resulting in low arithmetic intensity for its attention operations, which are therefore mapped to PIMs. For the TLM, each iteration verifies a variable number of tokens per request. By pooling tokens before verification, the FC operations become consistently compute-bound and are scheduled onto xPUs.

In contrast, whether the DLM's FC operations and the TLM's attention operations are scheduled to PIMs or xPUs is determined dynamically.

After each prediction, the Scheduler identifies which requests are eligible for additional DLM iterations and uses this to determine the effective micro-batch size. Fluctuations in the effective micro-batch size directly affect the arithmetic intensity of the DLM's FC operator. The Scheduler quickly approximates this intensity and compares it against pre-characterized thresholds—PIM compute-bound and GPU memory-bound—to decide whether the operator should be remapped.

Before the verification stage begins, the Scheduler also counts the number of draft tokens per request in the Shared Pool to estimate the arithmetic intensity of the TLM's attention operator. It then applies the same decision process as for the DLM's FC operator to determine the most suitable execution engine.

At runtime, the Scheduler dynamically remaps operators in response to variations in micro-batch size and draft length, enabling the system to consistently select the most efficient execution engine under any workload [51], [52].

V. Evaluation

In this section, we evaluate the efficiency and effectiveness of SADDLE compared to state-of-the-art approaches.

A. Experimental Setup

Benchmarks. We evaluate three transformer-based LLMs as TLMs: Llama-3.1-70B-Instruct [47], OPT-66B [61], and OPT-175B [61], each paired with a corresponding DLM from the same model family: Llama-3.2-1B-Instruct, OPT-1.3B [61], and OPT-6.7B [61], respectively. The model configurations are summarized in Table I. All models use the FP16 data type, which is standard for inference tasks.

We use the Dolly dataset [7], an open-source, instruction-following dataset created by thousands of employees, spanning several behavior categories defined in InstructGPT [37]. To characterize system load, we vary batch sizes from 16 to 128 and set the maximum sequence length to 1024 tokens in most experiments. Due to memory constraints, the maximum sequence length for OPT-175B is limited to 512 tokens.

Baselines. We compare SADDLE against four state-of-the-art baselines: (1) **GPU-AD** [2]: Autoregressive decoding using the TLM on GPUs. (2) **GPU-SD** [2]: Speculative decoding on GPUs. (3) **PIM-AD** [38]: Autoregressive decoding on a heterogeneous PIM–GPU system, using the TLM. (4) **PIM-SD** [22]: Speculative decoding on a heterogeneous PIM–GPU system.

The two GPU baselines are evaluated on the A100 DGX system [32], which consists of eight A100 GPUs, each with 80 GB of memory, totaling 640 GB. The system delivers an aggregate memory bandwidth of 16 TB/s. All GPU baselines are implemented using DeepSpeed Inference [2].

The two PIM baselines are based on the HBM-PIM architecture used in AttAcc [38], where each memory bank is paired with one PE. These baselines are configured with the same number of GPUs as the DGX setup and feature 40 HBM stacks, each with 16 GB of memory, also totaling 640 GB. The internal memory bandwidth is 144 TB/s—nine times that of the DGX system. For PIM-AD, attention operators are offloaded to the PIM while FC operators are executed on the GPU. For PIM-SD, we follow the operator mapping strategy of SpecPIM [22], which performs design-space exploration before execution based on the initial batch size and maximum sequence length.

Configurations. We adopt the NVIDIA A100 GPU as the centralized processor for SADDLE. All HBM modules used in our experiments are HBM3 [18], operating at 5.2 Gbps per pin. For PIM PEs, we follow the same design as in PIM-AD and PIM-SD, in which each PE is placed near an HBM bank.

To ensure a fair comparison, each SADDLE device is provisioned with five HBM stacks connected via NVLink, each offering 16 GB of memory, totaling 80 GB—matching the memory capacity of an A100 GPU. We deploy eight SADDLE PIM devices in total, resulting in an aggregate memory capacity equivalent to the PIM baselines.

For the SADDLE Manager, we provision a 1 KB Shared Pool and a 1 KB Eager Pool, with the latter subdivided by the number of micro-batches, allowing each pool to store up to 512 tokens. Additionally, a 1 KB SRAM is allocated to store logit values and cumulative acceptance probabilities.

Simulation. We develop a cycle-accurate simulator by modifying Ramulator2 [27] and ATTACC [38] to evaluate the performance and energy efficiency of both GPU systems and SADDLE. The simulator takes system configuration and model specifications as input and outputs the execution time and energy consumption for each system.

To assess area and energy overhead, we synthesize the PEs using Synopsys Design Compiler with a 28 nm technology node at a 1 GHz clock frequency. The area overhead is

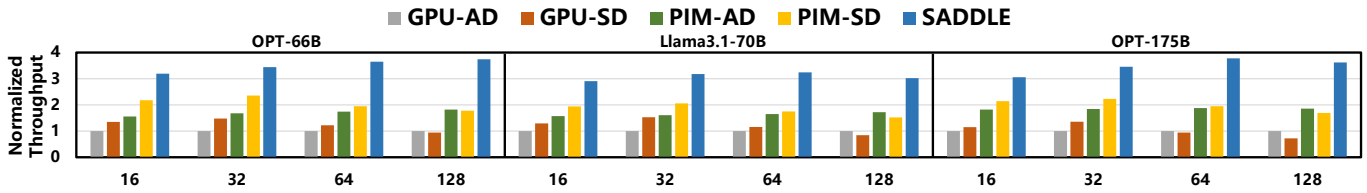

Fig. 12. Throughput of SADDLE compared to four baselines across models and batch sizes (normalized to GPU-AD), highlighting SADDLE's consistent performance gains

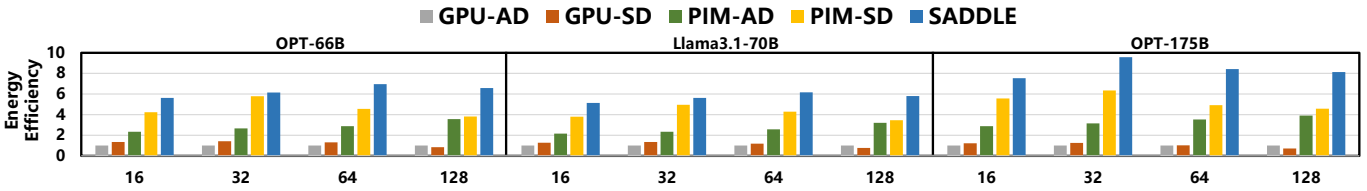

Fig. 13. Energy efficiency of SADDLE compared to four baselines across models and batch sizes (normalized to GPU-AD), demonstrating SADDLE's consistent energy advantages

TABLE I
MODEL CONFIGURATIONS

Model	#Parameters	d_{model}	#Layers	#Heads
OPT	1.3B	2048	24	32
	6.7B	4096	32	32
	66B	9216	64	72
	175B	12288	96	96
Llama3	1B	2048	16	32
	70B	8192	80	64

scaled to the DRAM process [9]. For HBM energy modeling, we reference activation and read energy values from prior work [34].

B. Overall Performance

Throughput. As shown in Figure 12, SADDLE consistently achieves the highest throughput across all workloads. Compared to GPU-AD, GPU-SD, PIM-AD, and PIM-SD, it improves average throughput by 3.36×, 2.88×, 1.94×, and 1.71×, respectively.

At batch sizes of 16 and 32, both GPU/PIM-SD and SADDLE outperform GPU/PIM-AD, showing that under light workloads, speculative decoding benefits from its predictive mechanism. However, as batch size increases, the performance advantage of GPU/PIM-SD over GPU/PIM-AD diminishes or even reverses. In contrast, SADDLE sustains a clear performance lead, demonstrating its effectiveness under heavier workloads. These gains arise from several factors. First, SADDLE adaptively adjusts the number of draft tokens per request at runtime, avoiding the computational waste inherent in fixed-length speculative decoding. When optimal draft lengths vary significantly across requests, this fine-grained control reduces unnecessary computation, resulting in higher effective throughput—especially evident under larger batch sizes.

Second, SADDLE mitigates the imbalance caused by variable draft lengths through cross-micro-batch verification, eliminating pipeline stalls between prediction and verification. This enables parallel execution of the DLM and TLM, unlocking greater acceleration potential.

Third, SADDLE dynamically maps each operator to either PIM or GPU based on its arithmetic intensity and bandwidth requirements, further boosting overall system efficiency.

Energy Efficiency. Figure 13 characterizes the energy efficiency achieved by SADDLE compared to our baselines. Specifically, SADDLE improves average energy efficiency by 6.81×, 5.96×, 2.32×, and 1.45× compared to GPU-AD, GPU-SD, PIM-AD, and PIM-SD, respectively. Compared to GPU-AD/SD, SADDLE offloads memory-bound operators to the PIM chip, thereby avoiding the high energy cost of frequently transferring intermediate activation matrices to GPU global memory. Relative to PIM-AD, although speculative decoding introduces additional FLOPs, overall energy consumption decreases due to significantly reduced global memory accesses. Compared to PIM-SD, SADDLE further lowers energy overhead by eliminating redundant computations for invalid tokens. In summary, the performance gains delivered by SADDLE directly translate into substantial improvements in energy efficiency.

The impact of PIM on power consumption is twofold. On the one hand, PIM substantially reduces data movement between the HBM and the GPU for memory-bound operators, thereby lowering overall device power. On the other hand, DRAM access power scales with the number of concurrently accessed banks, and bank-level parallel accesses issued by all PEs account for the majority of power consumption. In our design, the TLM attention and DLM FC operators exploit token-level and request-level data reuse to increase arithmetic intensity, thereby reducing the number of bank accesses and mitigating DRAM access power. As a result, the overall device power remains within the assumed thermal design envelope.

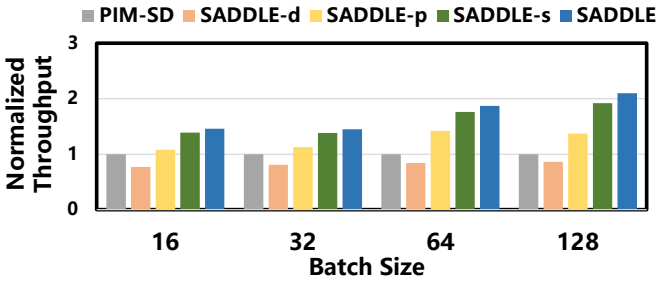

Fig. 14. Throughput of SADDLE (OPT-66B+OPT-1.3B) and its variants across batch sizes (normalized to PIM-SD), highlighting gains from adaptive length, shared pool, eager pool, and dynamic mapping

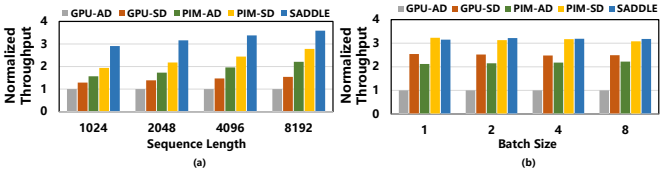

Fig. 15. Throughput sensitivity of SADDLE to (a) sequence length and (b) batch size, showing increasing gains with longer sequences and competitive performance under small batches

C. Ablation Analysis

To assess the contribution of each design component to overall performance, we evaluate several SADDLE variants on OPT-66B and OPT-1.3B. These include: using only the adaptive draft length strategy (SADDLE-d); combining adaptive draft length with the Shared Pool for cross-micro-batch verification (Section IV-C), while still executing the DLM and TLM sequentially (SADDLE-p); and a configuration that adds the eager pool mechanism (Section IV-D) while mapping FC operators to the GPU and attention operators (Section IV-E) to PIM (SADDLE-s). Figure 14 presents the performance of these variants, normalized to PIM-SD.

We observe that applying only adaptive draft length (SADDLE-d) results in an average throughput reduction of 1.22× compared to PIM-SD. This highlights that adaptive drafting alone does not improve performance due to inter-stage pipeline bubbles (Figure 5(b)) and fluctuating operator intensity (Figure 6). Introducing the Shared Pool for cross-micro-batch verification in SADDLE-p improves performance by 1.52× over SADDLE-d and by 1.25× over PIM-SD on average. Incorporating the eager pool mechanism yields an additional 1.24× speedup. Finally, SADDLE-s, which dynamically maps operators to GPUs or PIMs, achieves a further 1.13× improvement.

These results confirm each component's contribution to SADDLE's performance gains.

D. Sensitivity Analysis

Sequence Length. LLMs are trending toward longer sequence lengths to capture extended contexts. Figure 15(a) shows SADDLE's performance on Llama3.1-70B+Llama3.2-1B for decoding lengths from 1024 to 8192 with batch size 16, normalized to GPU-AD. SADDLE's advantage grows with longer sequences, as the attention layer's execution time increases due to the expanded KV cache. Offloading attention to PIM yields greater performance gains. Overall, SADDLE consistently outperforms across varying sequence lengths.

Batch Size. Figure 15(b) shows SADDLE's performance on OPT-66B+OPT-1.3B for small batch sizes (1, 2, 4 and 8), normalized to GPU-AD. Under light loads, SADDLE matches PIM-SD as wasted computation has minimal impact, while still outperforming other baselines, demonstrating its robustness even at low system utilization.

E. Performance Discussion

Latency Breakdown. Figure 16 shows the breakdown of the end-to-end inference latency of PIM-SD and SADDLE. We observe that only 0.83% of SADDLE's end-to-end latency is spent in monitoring and decision-making. This overhead is negligible because monitoring and decision making only require a few multiplications to update $H_t = \prod_{i=1}^{t} p_i$ and a simple comparison $H_t > \tau$, both implemented as dedicated hardware modules in the SADDLE Manager. By adaptively adjusting the draft length, redundant computation and parallelism degradation are effectively mitigated. SADDLE reduces the prediction and verification latency by 1.18× and 1.23×, respectively, compared to PIM-SD. Furthermore, the decoupled asynchronous pipeline overlaps prediction and verification, achieving 1.73× end-to-end latency reduction.

Communication Costs. Figure 17 breaks down the the execution time of SADDLE's TLM verification phase. We see that only 13.54% of the TLM verification time is spent in cross-pCH communication. The reasons are twofold. First, accumulators are placed on each DRAM die to locally aggregate partial results from different bank groups, thereby reducing the volume of data transmitted across pCHs. Second, the high internal memory bandwidth further mitigates communication latency. Similarly, cross-stack communication accounts for 7.51% and does not become a performance bottleneck due to the following two reasons. First, we integrate an SFU on the buffer die of each HBM stack to aggregate partial results from different pCHs, effectively minimizing cross-stack data movement volume. Second, high internal (2 TB/s intra-device) and inter-device (600 GB/s) bandwidths further hide the cross-stack communication latency.

Hardware Utilization. A key contributor to SADDLE's throughput improvement is its higher hardware resource utilization across both the GPU and PIM subsystems. Figure 18 depicts the average utilization of GPU and PIM for PIM-AD, PIM-SD, and SADDLE at a batch size of 64 across three models. Compared with PIM-AD and PIM-SD, SADDLE improves the GPU utilization by 1.13× and 1.37×, and enhances the PIM utilization by 1.84× and 1.18×, respectively. These gains arise from SADDLE's asynchronous decoding pipeline, which effectively alleviates pipeline stalls.

As shown in Figure 19, without arithmetic intensity-aware operator scheduling, SADDLE executes 9.51% of operations

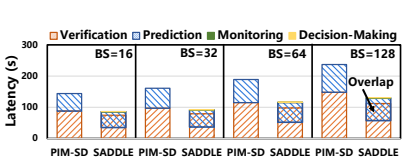

Fig. 16. Latency breakdown of PIM-SD and SADDLE

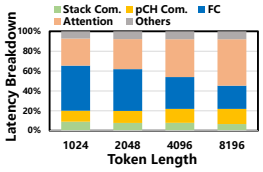

Fig. 17. The execution time breakdown of SADDLE's TLM verification phase

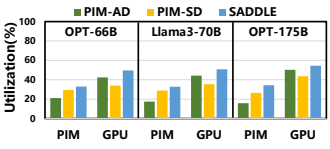

Fig. 18. The PIM and GPU utilization of SADDLE and baselines across models

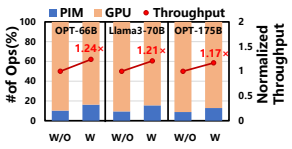

Fig. 19. The FLOPS breakdown and the throughput of SADDLE without and with operator scheduling

on PIM and 90.49% on the GPU, respectively. After enabling operator scheduling, these proportions shift to 14.89% and 85.11%, yielding a 1.21× throughput improvement by fully exploiting the acceleration potential of the heterogeneous architecture.

F. Area Overhead

SADDLE introduces an area overhead of approximately $16.24\,\mathrm{mm}^2$ per DRAM die and $1.62\,\mathrm{mm}^2$ per buffer die in an HBM stack. Each pCH integrates 16 PEs and 4 accumulators. In a 1z-nm DRAM process [39], [43], a PE occupies $0.116\,\mathrm{mm}^2$ and an accumulator $0.044\,\mathrm{mm}^2$. PE area is split among arithmetic units (57%), on-chip buffers (16%), and control logic (27%), while accumulator area is dominated by arithmetic. Our PE control logic is specialized for matrix-vector products, keeping it compact compared to prior PIM designs supporting general operations. Given a $121\,\mathrm{mm}^2$ HBM3 die, our PIM logic contributes 13.4% DRAM-die area overhead, which is comparable to prior HBM-PIM accelerators [15], [38]. Therefore, in our design, the additional computational logic does not compromise memory capacity. On the 7 nm buffer die, the softmax accelerator and its accumulator occupy $1.44\,\mathrm{mm}^2$ and $0.18\,\mathrm{mm}^2$, respectively, with most area attributed to buffering in the softmax unit.

VI. Related Work

ASIC-based LLM Accelerators: A^3 [13] accelerates attention by identifying key connections and applying top-k approximation to handle long sequences. SpAtten [49] optimizes different Transformer stages using cascaded tokens, head pruning, and progressive quantization to exploit token/head sparsity and quantization. Sanger [26] predicts sparse attention with low-precision computation and improves hardware efficiency via pack and split methods. ELSA [14] computes input hashes at runtime to find the most similar keys based on hash distance and restricts attention to these pairs. DOTA [41] introduces a dynamic sparse attention predictor that learns attention weight patterns via an approximate detector. These approaches reduce attention computation via sparsification (e.g., top-k, head pruning), low-precision quantization, hash-based filtering, and dynamic sparsity prediction. However, as they target only the attention layer, they struggle to deliver high end-to-end speedups under large batch sizes.

PIM-based LLM Accelerators: PIM brings compute kernels like GEMV and attention closer to DRAM banks, reducing off-chip transfers and latency. With processing elements near memory arrays, PIM leverages on-chip bandwidth far exceeding that of standard DRAM channels, making it ideal for memory-bound workloads like autoregressive decoding, where KV cache size scales with context length and batch size. TransPIM [64] pioneers memory-centric transformer inference by placing QKV tiles in adjacent banks and computing attention entirely in DRAM, while offloading softmax and residual paths to the host via an interposer. CENT [11] links CXL-attached PIM modules in a GPU-free fabric, promoting modular near-bank compute with memory expansion as a cost-effective scaling strategy. However, PIM-only designs fall short in accelerating compute-bound FC layers.

PIM-Enabled Heterogeneous Systems for LLM Inference: AttAcc [38] offloads attention to HBM-PIM while retaining FC operators on the GPU, aligning operator placement with bandwidth and compute demands to support batched generation. IANUS [44] combines an NPU and PIM behind a shared memory fabric, avoiding explicit copies but limiting concurrency between normal accesses and PIM execution, favoring low-latency over throughput. SpecPIM [22] addresses speculative decoding via offline scheduling using genetic algorithms and Monte Carlo Tree Search to optimize task placement per batch size and draft length, though it lacks adaptability at runtime. PAPI [15] dynamically profiles operator intensity and reallocates workloads between GPU and PIM to minimize idle time as decoding progresses. These systems highlight the trend of dynamic scheduling based on workload behavior. SADDLE advances this line by adaptively orchestrating speculative decoding end-to-end, achieving higher throughput.

Adaptive Draft Sequence Length: The concept of adaptive draft sequence length has been explored in prior work [28], [50]. Disco [28] employs a lightweight classifier to decide, after generating each draft token, whether to continue drafting or switch to verification. OPT-Tree [50], under a given node budget, performs a stepwise greedy search to construct an optimal draft tree that maximizes the expected accepted length. These studies demonstrate that adaptive draft length methods effectively reduce latency in single-request scenarios. However, a long-standing practical challenge lies in enabling large-batch speculative decoding. SADDLE employs a lightweight, hardware-friendly adaptive draft length method that is plug-and-play and requires no additional training. More importantly, SADDLE integrates adaptive drafting with batching and introduces a novel prediction–verification decoupled asynchronous pipeline to mitigate pipeline stalls exacerbated by varying draft lengths across requests within a batch. This enables

the hardware to achieve significant throughput gains with speculative decoding, rather than only reducing latency for a single batch. Advanced adaptive drafting algorithms can also be incorporated into SADDLE, as they are orthogonal and complementary to our system-level design.

VII. Conclusion

Speculative decoding enhances LLM throughput but suffers from redundant computation and limited parallelism on PIM-enabled heterogeneous systems, particularly under large batch sizes. To address this, we present SADDLE, a PIM-enabled heterogeneous system that adaptively adjusts draft sequence lengths and overcomes key performance bottlenecks. SADDLE incorporates three core optimizations: (1) runtime draft-length tuning based on cumulative acceptance probability to reduce invalid tokens and maintain TLM parallelism, (2) an asynchronous decoding pipeline that decouples DLM and TLM to avoid pipeline stalls, and (3) arithmetic intensity–aware operator scheduling that dynamically maps tasks to PIM or GPU for efficient hardware utilization. Experimental results show that SADDLE improves inference throughput by 2.88× over GPU-only and 1.71× over existing PIM-enabled speculative decoding systems on average.

Acknowledgements

This work is supported by the National Key Research and Development Program of China under Grant No. 2023YFB4503400 and the National Natural Science Foundation of China (Nos. 62402456, 62450064, and 62322205). The correspondence of this paper should be addressed to Qinggang Wang.

References

[1] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, "Taming throughput-latency tradeoff in LLM inference with sarathi-serve," in *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024, pp. 117–134.

[2] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Li, E. Zheng, O. Ruwase, S. Smith, M. Zhang, J. Rasley, and Y. He, "Deepspeed-inference: Enabling efficient inference of transformer models at unprecedented scale," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2022, pp. 46:1–46:15.

[3] R. Anil, A. M. Dai, O. Firat, M. Johnson, D. Lepikhin, A. Passos, S. Shakeri, E. Taropa, P. Bailey, Z. Chen, E. Chu, J. H. Clark, L. E. Shafey, Y. Huang, K. Meier-Hellstern, G. Mishra, E. Moreira, M. Omernick, K. Robinson, S. Ruder, Y. Tay, K. Xiao, Y. Xu, Y. Zhang, G. H. Abrego, J. Ahn, J. Austin, P. Barham, J. Botha, J. Bradbury, S. Brahma, K. Brooks, M. Catasta, Y. Cheng, C. Cherry, C. A. Choquette-Choo, A. Chowdhery, C. Crepy, S. Dave, M. Dehghani, S. Dev, J. Devlin, M. Díaz, N. Du, E. Dyer, V. Feinberg, F. Feng, V. Fienber, M. Freitag, X. Garcia, S. Gehrmann, L. Gonzalez, G. Gur-Ari, S. Hand, H. Hashemi, L. Hou, J. Howland, A. Hu, J. Hui, J. Hurwitz, M. Isard, A. Ittycheriah, M. Jagielski, W. Jia, K. Kenealy, M. Krikun, S. Kudugunta, C. Lan, K. Lee, B. Lee, E. Li, M. Li, W. Li, Y. Li, J. Li, H. Lim, H. Lin, Z. Liu, F. Liu, M. Maggioni, A. Mahendru, J. Maynez, V. Misra, M. Moussalem, Z. Nado, J. Nham, E. Ni, A. Nystrom, A. Parrish, M. Pellat, M. Polacek, A. Polozov, R. Pope, S. Qiao, E. Reif, B. Richter, P. Riley, A. C. Ros, A. Roy, B. Saeta, R. Samuel, R. Shelby, A. Slone, D. Smilkov, D. R. So, D. Sohn, S. Tokumine, D. Valter, V. Vasudevan, K. Vodrahalli, X. Wang, P. Wang, Z. Wang, T. Wang, J. Wieting, Y. Wu, K. Xu, Y. Xu, L. Xue, P. Yin, J. Yu, Q. Zhang, S. Zheng, C. Zheng, W. Zhou, D. Zhou, S. Petrov, and Y. Wu, "Palm 2 technical report," *arXiv preprint arXiv:2305.10403*, 2023.

[4] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, D. Chen, and T. Dao, "Medusa: Simple LLM inference acceleration framework with multiple decoding heads," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

[5] C. Chen, S. Borgeaud, G. Irving, J. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," *arXiv preprint arXiv:2302.01318*, 2023.

[6] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[7] M. Conover, M. Hayes, A. Mathur, J. Xie, J. Wan, S. Shah, A. Ghodsi, P. Wendell, M. Zaharia, and R. Xin, "Free dolly: Introducing the world's first truly open instruction-tuned llm," 2023. [Online]. Available: https://github.com/databrickslabs/dolly

[8] D. Dai, C. Deng, C. Zhao, R. X. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, Z. Xie, Y. K. Li, P. Huang, F. Luo, C. Ruan, Z. Sui, and W. Liang, "Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 1280–1297.

[9] F. Devaux, "The true processing in memory accelerator," in *Proceedings of the 2019 IEEE Hot Chips Symposium (HCS)*, 2019, pp. 1–24.

[10] Google, "Gemini," 2025. [Online]. Available: https://gemini.google.com/

[11] Y. Gu, A. Khadem, S. Umesh, N. Liang, X. Servot, O. Mutlu, R. R. Iyer, and R. Das, "PIM is all you need: A cxl-enabled gpu-free system for large language model inference," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025, pp. 862–881.

[12] M.-H. Guo, J. Xu, Y. Zhang, J. Song, H. Peng, Y.-X. Deng, X. Dong, K. Nakayama, Z. Geng, C. Wang, B. Ni, G.-W. Yang, Y. Rao, H. Peng, H. Hu, G. Wetzstein, and S.-M. Hu, "R-bench: Graduate-level multi-disciplinary benchmarks for llm & mllm complex reasoning evaluation," *arXiv preprint arXiv:2505.02018*, 2025.

[13] T. J. Ham, S. Jung, S. Kim, Y. H. Oh, Y. Park, Y. Song, J. Park, S. Lee, K. Park, J. W. Lee, and D. Jeong, "A^3: Accelerating attention mechanisms in neural networks with approximation," in *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2020, pp. 328–341.

[14] T. J. Ham, Y. Lee, S. H. Seo, S. Kim, H. Choi, S. J. Jung, and J. W. Lee, "ELSA: hardware-software co-design for efficient, lightweight self-attention mechanism in neural networks," in *Proceedings of the 48th ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 692–705.

[15] Y. He, H. Mao, C. Giannoula, M. Sadrosadati, J. Gómez-Luna, H. Li, X. Li, Y. Wang, and O. Mutlu, "PAPI: exploiting dynamic parallelism in large language model decoding with a processing-in-memory-enabled computing system," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025, pp. 766–782.

[16] G. Heo, S. Lee, J. Cho, H. Choi, S. Lee, H. Ham, G. Kim, D. Mahajan, and J. Park, "Neupims: NPU-PIM heterogeneous acceleration for batched LLM inferencing," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024, pp. 722–737.

[17] S. Hong, S. Moon, J. Kim, S. Lee, M. Kim, D. Lee, and J. Kim, "DFX: A low-latency multi-fpga appliance for accelerating transformer-based text generation," in *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 616–630.

[18] JEDEC, "High bandwidth memory dram (hbm3)," 2022.

[19] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.

[20] S. Lee, S. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin, J. Kim, S. O, A. Iyer, D. Wang, K. Sohn, and N. S. Kim, "Hardware architecture and software stack for PIM based on commercial DRAM technology: Industrial product," in *Proceedings of the 48th ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 43–56.

[21] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2023, pp. 19 274–19 286.

[22] C. Li, Z. Zhou, S. Zheng, J. Zhang, Y. Liang, and G. Sun, "Specpim: Accelerating speculative inference on pim-enabled system via architecture-dataflow co-exploration," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024, pp. 950–965.

[23] K. Li, W. Huang, Q. Wang, L. Zheng, X. Liao, H. Jin, and J. Xue, "Diff-moe: Efficient batched moe inference with priority-driven differential expert caching," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2025, pp. 1951–1965.

[24] T. Liu, Y. Li, Q. Lv, K. Liu, J. Zhu, W. Hu, and X. Sun, "PEARL: parallel speculative decoding with adaptive draft length," in *Proceedings of the Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

[25] X. Liu, J. Park, L. Hu, W. Kwon, Z. Li, C. Zhang, K. Du, X. Mo, K. You, A. Cheung, Z. Deng, I. Stoica, and H. Zhang, "Turbospec: Closed-loop speculation control system for optimizing llm serving goodput," *arXiv preprint arXiv:2406.14066*, 2025.

[26] L. Lu, Y. Jin, H. Bi, Z. Luo, P. Li, T. Wang, and Y. Liang, "Sanger: A co-design framework for enabling sparse attention using reconfigurable architecture," in *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2021, pp. 977–991.

[27] H. Luo, Y. C. Tuğrul, F. N. Bostancı, A. Olgun, A. G. Yağlıkçı, and O. Mutlu, "Ramulator 2.0: A modern, modular, and extensible dram simulator," *IEEE Computer Architecture Letters*, vol. 23, no. 1, pp. 112–116, 2024.

[28] J. Mamou, O. Pereg, D. Korat, M. Berchansky, N. Timor, M. Wasserblat, and R. Schwartz, "Dynamic speculation lookahead accelerates speculative decoding of large language models," in *Proceedings of the NeurIPS Workshop on Efficient Natural Language and Speech Processing (ENLSP)*, vol. 262, 2024, pp. 456–467.

[29] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi, C. Shi, Z. Chen, D. Arfeen, R. Abhyankar, and Z. Jia, "Specinfer: Accelerating large language model serving with tree-based speculative inference and verification," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems(ASPLOS)*, 2024, pp. 932–949.

[30] D. Narayanan, M. Shoeybi, J. Casper, P. LeGresley, M. Patwary, V. Korthikanti, D. Vainbrand, P. Kashinkunti, J. Bernauer, B. Catanzaro, A. Phanishayee, and M. Zaharia, "Efficient large-scale language model training on GPU clusters using megatron-lm," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2021, pp. 58:1–58:15.

[31] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "Codegen: An open large language model for code with multi-turn program synthesis," in *Proceedings of the Eleventh International Conference on Learning Representations (ICLR)*, 2023.

[32] NVIDIA, "NVIDIA DGX A100," 2023. [Online]. Available: https://resources.nvidia.com/enus-dgx-systems/dgx-ai

[33] NVIDIA, "Nvlink and nvlink switch," 2025. [Online]. Available: https://www.nvidia.com/en-us/data-center/nvlink/

[34] M. O'Connor, N. Chatterjee, D. Lee, J. M. Wilson, A. Agrawal, S. W. Keckler, and W. J. Dally, "Fine-grained DRAM: energy-efficient DRAM for extreme bandwidth systems," in *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2017, pp. 41–54.

[35] OpenAI, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2024.

[36] OpenAI, "Chatgpt," 2025. [Online]. Available: https://openai.com/blog/chatgpt

[37] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. F. Christiano, J. Leike, and R. Lowe, "Training language models to follow instructions with human feedback," in *Proceedings of the 36th International Conference on Neural Information Processing Systems (NeurIPS)*, 2022, pp. 27 730–27 744.

[38] J. Park, J. Choi, K. Kyung, M. J. Kim, Y. Kwon, N. S. Kim, and J. H. Ahn, "Attacc! unleashing the power of PIM for batched transformer-based generative model inference," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024, pp. 103–119.

[39] M.-J. Park, J. Lee, K. Cho, J. Park, J. Moon, S.-H. Lee, T.-K. Kim, S. Oh, S. Choi, Y. Choi, H. S. Cho, T. Yun, Y. J. Koo, J.-S. Lee, B.-K. Yoon, Y.-J. Park, S. Oh, C. K. Lee, S.-H. Lee, H.-W. Kim, Y. Ju, S.-K. Lim, K. Y. Lee, S.-H. Lee, W. S. We, S. Kim, S. M. Yang, K. Lee, I.-K. Kim, Y. Jeon, J.-H. Park, J. C. Yun, S. Kim, D.-Y. Lee, S.-H. Oh, J.-H. Shin, Y. Lee, J. Jang, and J. Cho, "A 192-gb 12-high 896-gb/s hbm3 dram with a tsv auto-calibration scheme and machine-learning-based layout optimization," *IEEE Journal of Solid-State Circuits*, vol. 58, no. 1, pp. 256–269, 2023.

[40] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient generative LLM inference using phase splitting," in *Proceedings of the 51st ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 118–132.

[41] Z. Qu, L. Liu, F. Tu, Z. Chen, Y. Ding, and Y. Xie, "DOTA: detect and omit weak attentions for scalable transformer acceleration," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2022, pp. 14–26.

[42] R. Ruiz-Dolz, Z. Kikteva, and J. Lawrence, "Mining complex patterns of argumentative reasoning in natural language dialogue," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025, pp. 7421–7435.

[43] Y. Ryu, S.-G. Ahn, J. H. Lee, J. Park, Y. K. Kim, H. Kim, Y. G. Song, H.-W. Cho, S. Cho, S. H. Song, H. Lee, U. Shin, J. Ahn, J.-M. Ryu, S. Lee, K.-H. Lim, J. Lee, J. H. Park, J.-S. Jeong, S. Joo, D. Cho, S. Y. Kim, M. Lee, H. Kim, M. Kim, J.-S. Kim, J. Kim, H. G. Kang, M.-K. Lee, S.-R. Kim, Y.-C. Kwon, Y. Y. Byun, K. Lee, S. Park, J. Youn, M.-O. Kim, K. Sohn, S.-J. Hwang, and J. Lee, "A 16 gb 1024 gb/s hbm3 dram with source-synchronized bus design and on-die error control scheme for enhanced ras features," *IEEE Journal of Solid-State Circuits*, vol. 58, no. 4, pp. 1051–1061, 2023.

[44] M. Seo, X. T. Nguyen, S. J. Hwang, Y. Kwon, G. Kim, C. Park, I. Kim, J. Park, J. Kim, W. Shin, J. Won, H. Choi, K. Kim, D. Kwon, C. Jeong, S. Lee, Y. Choi, W. Byun, S. Baek, H. Lee, and J. Kim, "IANUS: integrated accelerator based on NPU-PIM unified memory system," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024, pp. 545–560.

[45] D. D. Sharma, R. Blankenship, and D. S. Berger, "An introduction to the compute express link (cxl) interconnect," *ACM Computing Surveys*, vol. 56, no. 11, pp. 290:1–290:37, 2024.

[46] B. Spector and C. Re, "Accelerating llm inference with staged speculative decoding," *arXiv preprint arXiv:2308.04623*, 2023.

[47] L. Team, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[48] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS)*, 2017, pp. 5998–6008.

[49] H. Wang, Z. Zhang, and S. Han, "Spatten: Efficient sparse attention architecture with cascade token and head pruning," in *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2021, pp. 97–110.

[50] J. Wang, Y. Su, J. Li, Q. Xia, Z. Ye, X. Duan, Z. Wang, and M. Zhang, "Opt-tree: Speculative decoding with adaptive draft tree structure," *Transactions of the Association for Computational Linguistics*, vol. 13, pp. 188–199, 2025.

[51] Q. Wang, L. Zheng, Z. An, H. Huang, H. Zhu, Y. Huang, P. Yao, X. Liao, and H. Jin, "High-performance and resource-efficient dynamic memory management in high-level synthesis," in *Proceedings of the 61st ACM/IEEE Design Automation Conference (DAC)*, 2024, pp. 203:1–203:6.

[52] Q. Wang, L. Zheng, Z. An, S. Xiong, R. Wang, Y. Huang, P. Yao, X. Liao, H. Jin, and J. Xue, "A scalable, efficient, and robust dynamic memory management library for hls-based fpgas," in *Proceedings of*

the 57th IEEE/ACM International Symposium on Microarchitecture (MICRO), 2024, pp. 437–450.

[53] Q. Wang, L. Zheng, A. Hu, Y. Huang, P. Yao, C. Gui, X. Liao, H. Jin, and J. Xue, "A data-centric accelerator for high-performance hypergraph processing," in *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 1326–1341.

[54] Q. Wang, L. Zheng, Y. Huang, P. Yao, C. Gui, X. Liao, H. Jin, W. Jiang, and F. Mao, "Grasu: A fast graph update library for fpga-based dynamic graph processing," in *Proceedings of the 2021 ACM/SIGDA International Symposium on Field Programmable Gate Array (FPGA)*, 2021, pp. 149–159.

[55] Q. Wang, L. Zheng, J. Yuan, Y. Huang, P. Yao, C. Gui, A. Hu, X. Liao, and H. Jin, "Hardware-accelerated hypergraph processing with chain-driven scheduling," in *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2022, pp. 184–198.

[56] Q. Wang, L. Zheng, J. Zhao, X. Liao, H. Jin, and J. Xue, "A conflict-free scheduler for high-performance graph processing on multi-pipeline fpgas," *ACM Transactions on Architecture and Code Optimization*, vol. 17, no. 2, pp. 14:1–14:26, 2020.

[57] S. Wang, H. Yang, X. Wang, T. Liu, P. Wang, Y. Xu, X. Liang, K. Ma, T. Feng, X. You, R. Gong, R. Wang, Z. Luan, Y. Liu, and D. Qian, "Towards efficient llm inference via collective and adaptive speculative decoding," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2025, pp. 973–990.

[58] G. Yu, J. S. Jeong, G. Kim, S. Kim, and B. Chun, "Orca: A distributed serving system for transformer-based generative models," in *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022, pp. 521–538.

[59] S. Yun, K. Kyung, J. Cho, J. Choi, J. Kim, B. Kim, S. Lee, K. Sohn, and J. H. Ahn, "Duplex: A device for large language models with mixture of experts, grouped query attention, and continuous batching," in *Proceedings of the 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024, pp. 1429–1443.

[60] J. Zhang, J. Wang, H. Li, L. Shou, K. Chen, G. Chen, and S. Mehrotra, "Draft& verify: Lossless large language model acceleration via self-speculative decoding," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 11 263–11 282.

[61] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin, T. Mihaylov, M. Ott, S. Shleifer, K. Shuster, D. Simig, P. S. Koura, A. Sridhar, T. Wang, and L. Zettlemoyer, "Opt: Open pre-trained transformer language models," *arXiv preprint arXiv:2205.01068*, 2022.

[62] C. Zhao, C. Deng, C. Ruan, D. Dai, H. Gao, J. Li, L. Zhang, P. Huang, S. Zhou, S. Ma, W. Liang, Y. He, Y. Wang, Y. Liu, and Y. X. Wei, "Insights into deepseek-v3: Scaling challenges and reflections on hardware for AI architectures," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA)*, 2025, pp. 1731–1745.

[63] L. Zheng, Z. Li, H. Zhang, Y. Zhuang, Z. Chen, Y. Huang, Y. Wang, Y. Xu, D. Zhuo, E. P. Xing, J. E. Gonzalez, and I. Stoica, "Alpa: Automating inter- and intra-operator parallelism for distributed deep learning," in *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022, pp. 559–578.

[64] M. Zhou, W. Xu, J. Kang, and T. Rosing, "Transpim: A memory-based acceleration via software-hardware co-design for transformer," in *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2022, pp. 1071–1085.